

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_a48e6aa3-748\agent-a4c3bd21c5c4f1825.jsonl`  
- SHA-256: `7dd20bc48f98b840d5c4031a7dd615a0b1ced970f95e61dc3c897aff15dd00c7`  
- Source modified: `2026-06-21T16:37:18+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_a48e6aa3-748`  
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T16:31:36.792Z)

Review T4 for ACCESSIBILITY + non-tech UX + no REGRESSION. T4 (North Star keystone): in-UI local file UPLOAD wired to the engine's chunked-upload endpoints.

Files under C:/Users/avidu/Projects/khelsutra/web/src:

- api/client.ts: uploadInit/uploadPart(raw octet-stream PUT)/uploadComplete/uploadAbort + uploadFile(file,{cid,signal,onProgress}) orchestrator (init -> slice into max_part_mb parts -> sequential PUT 0,1,2 -> complete; best-effort DELETE on failure/abort).
- api/types.ts: UploadInit/Part/CompleteResponse.
- hooks/useUpload.ts: start(file) -> uploading{pct} -> onComplete(serverPath) | error(classifyError); rejects 0-byte; AbortController cancel/unmount.
- components/IngestPanel.tsx: file picker (primary) + the existing gs:// URI form (secondary); on upload complete -> useIngestJob.submit(path) (the SAME process->poll->onLoaded back-half); upload % progressbar; A8 chips reused.
- catalogs en/kn/hi: +ingest.chooseFile/uploadAria/fileHint/orLink/uploading.

Engine contract at C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py.
150 web tests green; tsc + build clean.

Read C:/Users/avidu/Projects/khelsutra/web/src/components/IngestPanel.tsx + the en catalog.

1. FILE PICKER a11y: the native input is sr-only inside a styled <label>; it has aria-label via t(). Is it keyboard-reachable + labelled for a screen reader? Is the visible "Choose a video file" button ?44px (A4)? Does the no-bare-strings hygiene test still pass (no hardcoded placeholder/aria-label/title)?
2. PROGRESS: the upload % uses role=progressbar + aria-valuenow/min/max + an icon+text chip (A8). Correct? Honest (determinate % for upload vs indeterminate for the engine phase)?
3. BOTH-PATHS: is the existing gs:// URI path preserved (so prior tests + users still work)? During busy, are BOTH inputs disabled and a single Cancel shown? Is the unified error (upload OR engine) surfaced via the ? chip + errorText, with retry resetting BOTH?
4. HANDOFF: on upload complete, does onLoaded ultimately receive the JOB's canonical video_id (from /api/process), NOT the upload's pre-hash video_id? (App lifts ?video=<id>.)
5. Copy honesty (en): fileHint/uploading/orLink ? clear for a non-tech user, no over-promises? Be precise. Empty findings if solid.

2. assistant (2026-06-21T16:31:40.948Z)

I'll review T4 (in-UI file upload). Let me start by reading the key files.

3. user (2026-06-21T16:31:41.897Z)

```
1      // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point  
the tool at a  
2      // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to  
the engine's  
3      // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both  
converge on  
4      // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
```

```

to App. Honest
5      // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6      // (done but no rallies) is shown, never silently treated as a load.
7      import { useState } from "react";
8      import type { ChangeEvent, FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11     import { useUpload } from "../hooks/useUpload";
12     import { errorText } from "../lib/errorText";
13     import { StatusChip, Spinner } from "../StatusChip";
14
15     interface Props {
16       /** Called with the engine's video_id once a submitted job finishes successfully. */
17       onLoaded: (videoId: string) => void;
18     }
19
20     export function IngestPanel({ onLoaded }: Props) {
21       const { t } = useTranslation();
22       const [uri, setUri] = useState("");
23       const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
24       // On upload completion, feed the returned server PATH straight into the engine
processing back-half.
25       const upload = useUpload((path) => submit(path));
26       const busy = ingestBusy || upload.busy;
27
28       const onSubmitUri = (e: FormEvent) => {
29         e.preventDefault();
30         submit(uri);
31       };
32       const onFile = (e: ChangeEvent<HTMLInputElement>) => {
33         const file = e.target.files?.[0];
34         if (file) upload.start(file);
35         e.target.value = ""; // let re-picking the same file fire change again
36       };
37       const cancelAll = () => {
38         upload.cancel();
39         cancelIngest();
40       };
41       const resetAll = () => {
42         upload.reset();
43         resetIngest();
44       };
45
46       // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
47       // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
48       // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
49       const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
50       const errorMsg = errorState ? errorText(t, errorState) : null;
51
52       return (
53         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
54           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
55           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
56
57           {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
58           <div className="mt-4">
59             <label

```

```

60         className={
61             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 " +
62             (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
63         }
64     >
65         {t("ingest.chooseFile")}
66         <input
67             type="file"
68             accept=".mp4, .mov, video/mp4, video/quicktime"
69             aria-label={t("ingest.uploadAria")}
70             disabled={busy}
71             onChange={onFile}
72             className="sr-only"
73         />
74     </label>
75     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
76 </div>
77
78     {/* SECONDARY: or point at a cloud bucket / host path. */}
79     <div className="mt-4 border-t border-black/5 pt-4">
80         <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
81         <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
82             <input
83                 type="text"
84                 value={uri}
85                 onChange={(e) => setUri(e.target.value)}
86                 placeholder={t("ingest.uriPlaceholder")}
87                 aria-label={t("ingest.uriAriaLabel")}
88                 disabled={busy}
89                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
90             />
91             <button
92                 type="submit"
93                 disabled={busy}
94                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
95             >
96                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
97             </button>
98         </form>
99         <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
100     </div>
101
102     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
103         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
104     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
105         {upload.phase.status === "uploading" && (
106             <div data-testid="upload-progress">
107                 <StatusChip tone="info" icon={<Spinner />} label={t("ingest.uploading", {
pct: upload.phase.pct })} />
108             <div
109                 className="mt-2 h-1.5 w-full max-w-sm overflow-hidden rounded-full bg-
black/10"
110                 role="progressbar"
111                 aria-valuenow={upload.phase.pct}
112                 aria-valuemin={0}
113                 aria-valuemax={100}
114             >
115                 <div className="h-full bg-green transition-all" style={{ width:

```

```

`${upload.phase.pct}%` }} />
116         </div>
117         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
118         <button
119             type="button"
120             onClick={cancelAll}
121             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
122             >
123             {t("ingest.cancel")}
124         </button>
125     </div>
126 )}
127 {ingestBusy && (
128     <div>
129         <StatusChip
130             tone="info"
131             icon={<Spinner />}
132             label={
133                 t("ingest.working") +
134                 (phase.status === "polling" && phase.progress
135                     ? ` · ${t("ingest.progress", { progress: phase.progress })}`
136                     : "")
137             }
138         />
139         {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
140         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
141         <button
142             type="button"
143             onClick={cancelAll}
144             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
145             >
146             {t("ingest.cancel")}
147         </button>
148     </div>
149 )}
150 {errorMsg && (
151     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
152         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
153         <p className="mt-2">{errorMsg}</p>
154         <button
155             type="button"
156             onClick={resetAll}
157             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
158             >
159             {t("ingest.retry")}
160         </button>
161     </div>
162 )}
163 </div>
164 </section>
165 );
166 }
167

```

4. user (2026-06-21T16:31:42.372Z)

```

1      // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2      // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing

```

```

3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETES the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string) => void) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26
27       const stop = useCallback(() => {
28         ctrlRef.current?.abort();
29         ctrlRef.current = null;
30       }, []);
31
32       // Abort any in-flight upload on unmount.
33       useEffect(() => stop, [stop]);
34
35       const start = useCallback(
36         (file: File) => {
37           // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38           if (!file || file.size === 0) {
39             setPhase({ status: "error", error: { code: "invalidUri" } });
40             return;
41           }
42
43           stop(); // supersede any prior upload
44           const ctrl = new AbortController();
45           ctrlRef.current = ctrl;
46           const cid = newCorrelationId();
47           setPhase({ status: "uploading", pct: 0, filename: file.name });
48           log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50           void (async () => {
51             try {
52               const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                   if (ctrl.signal.aborted) return;
57                   const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                   setPhase({ status: "uploading", pct, filename: file.name });
59                 },
60               });

```

```

61         if (ctrl.signal.aborted) return;
62         setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63         onCompleteRef.current(res.path);
64     } catch (err) {
65         // User-cancelled (abort) is silent, not an error.
66         if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67         setPhase({ status: "error", error: classifyError(err) });
68         log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69     }
70     })();
71     },
72     [stop],
73 );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

5. user (2026-06-21T16:31:42.646Z)

```

1     // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2     // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3     // correlation id (header + logs) so a journey is traceable end to end.
4     import { log, newCorrelationId } from "../lib/log";
5     import type {
6         RallySegment,
7         QueryResponse,
8         CompileResponse,
9         EngineConfig,
10        ProcessResponse,
11        JobStatus,
12        UploadInitResponse,
13        UploadPartResponse,
14        UploadCompleteResponse,
15    } from "../types";
16
17    const BASE = "/api";
18
19    export class ApiError extends Error {
20        readonly cid?: string;
21        readonly status?: number;
22        readonly body?: string;
23        constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
24            super(message);
25            this.name = "ApiError";
26            this.cid = opts.cid;
27            this.status = opts.status;

```

```

28     this.body = opts.body;
29     if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
30 }
31 }
32
33 interface RequestOpts {
34     method?: string;
35     body?: string;
36     headers?: Record<string, string>;
37     /** Correlation id to thread through this call; auto-generated if absent. */
38     cid?: string;
39 }
40
41 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
42     const cid = opts.cid ?? newCorrelationId();
43     const url = `${BASE}${path}`;
44     const method = opts.method ?? "GET";
45     log("info", "api.request", { cid, method, url });
46
47     let res: Response;
48     try {
49         res = await fetch(url, {
50             method,
51             body: opts.body,
52             headers: { "content-type": "application/json", "x-correlation-id": cid,
... (opts.headers ?? {}) },
53         });
54     } catch (err) {
55         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
56         log("error", "api.network_error", { cid, method, url, error: String(err) });
57         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
58     }
59
60     if (!res.ok) {
61         const body = await res.text().catch(() => "");
62         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
63         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
64     }
65
66     const data = (await res.json()) as T;
67     log("info", "api.response", { cid, method, url, status: res.status });
68     return data;
69 }
70
71 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
72     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
73     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
74     export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
75         return request<ProcessResponse>("/process", {
76             method: "POST",
77             body: JSON.stringify({ video_path: videoPath }),
78             cid,
79         });
80     }
81
82     /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
83     export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
84         return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});

```

```

85     }
86
87     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
88     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
89         const data = await request<QueryResponse>("/query", {
90             method: "POST",
91             body: JSON.stringify({ video_id: videoId }),
92             cid,
93         });
94         return data.play_segments ?? [];
95     }
96
97     /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
98     export async function compileReel(
99         videoId: string,
100         segmentIds: number[],
101         outputFilename: string,
102         cid?: string,
103     ): Promise<CompileResponse> {
104         return request<CompileResponse>("/compile", {
105             method: "POST",
106             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
107             cid,
108         });
109     }
110
111     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
112     export function highlightUrl(videoId: string, filename: string): string {
113         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
114     }
115
116     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
117     export async function getConfig(cid?: string): Promise<EngineConfig> {
118         return request<EngineConfig>("/config", { method: "GET", cid });
119     }
120
121     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
122     export function minShotCount(cfg: EngineConfig): number | undefined {
123         return cfg.stitching?.min_shot_count;
124     }
125
126     // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
127     // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
128     // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
129
130     /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
131     export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
132         return request<UploadInitResponse>("/upload/init", {
133             method: "POST",
134             body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
135             cid,
136         });
137     }
138

```



```

139     /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
140     export async function uploadPart(
141         uploadId: string,
142         index: number,
143         chunk: Blob,
144         cid?: string,
145         signal?: AbortSignal,
146     ): Promise<UploadPartResponse> {
147         const c = cid ?? newCorrelationId();
148         const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
149         log("info", "api.request", { cid: c, method: "PUT", url });
150         let res: Response;
151         try {
152             res = await fetch(url, {
153                 method: "PUT",
154                 body: chunk,
155                 headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
156                 signal,
157             });
158         } catch (err) {
159             log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
160             throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
161         }
162         if (!res.ok) {
163             const body = await res.text().catch(() => "");
164             log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
165             throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
166         }
167         log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
168         return (await res.json()) as UploadPartResponse;
169     }
170
171     /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
172     export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
173         return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
174             method: "POST",
175             body: JSON.stringify({ total_parts: totalParts }),
176             cid,
177         });
178     }
179
180     /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
181     export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
182         await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
183     }
184
185     export interface UploadProgress {
186         sentBytes: number;
187         totalBytes: number;
188     }
189
190     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
191     * On ANY failure or abort it best-effort DELETes the server-side partial, then
rethrows so the
192     * caller can classify it (the engine's 400/413 detail survives). Returns {path,

```

```

video_id}. */
193     export async function uploadFile(
194         file: File,
195         opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
196     ): Promise<UploadCompleteResponse> {
197         const cid = opts.cid ?? newCorrelationId();
198         const init = await uploadInit(file.name, file.size, cid);
199         const partBytes = Math.max(1, init.max_part_mb) * 1024 * 1024;
200         const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
201         log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
202         let sent = 0;
203         try {
204             for (let i = 0; i < totalParts; i++) {
205                 if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
206                 const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
207                 await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
208                 sent += chunk.size;
209                 opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
210             }
211             const done = await uploadComplete(init.upload_id, totalParts, cid);
212             log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
213             return done;
214         } catch (err) {
215             log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
216             await uploadAbort(init.upload_id, cid).catch(() => {});
217             throw err;
218         }
219     }
220

```

6. assistant (2026-06-21T16:31:51.365Z)

Now let me read the catalogs and supporting files.

7. user (2026-06-21T16:31:52.110Z)

```

web\src\i18n\index.ts
web\src\i18n\hygiene.test.ts
web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json
web\src\i18n\locales\hi\common.json

```

8. user (2026-06-21T16:31:52.845Z)

```

1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";

```

```

11 import { classifyError, type IngestError } from "../lib/errors";
12 import { log, newCorrelationId } from "../lib/log";
13 import type { JobStatus } from "../api/types";
14
15 const POLL_MIN_MS = 1000;
16 const POLL_MAX_MS = 5000;
17 const POLL_BACKOFF = 1.5;
18 const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20 export type IngestPhase =
21   | { status: "idle" }
22   | { status: "submitting" }
23   | { status: "polling"; progress: string | null }
24   | { status: "done"; videoId: string }
25   | { status: "error"; error: IngestError };
26
27 interface Run {
28   cancelled: boolean;
29   timer?: ReturnType<typeof setTimeout>;
30 }
31
32 export function useIngestJob(onLoaded?: (videoId: string) => void) {
33   const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34   const runRef = useRef<Run | null>(null);
35   // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36   const onLoadedRef = useRef(onLoaded);
37   onLoadedRef.current = onLoaded;
38
39   const stop = useCallback(() => {
40     const run = runRef.current;
41     if (run) {
42       run.cancelled = true;
43       if (run.timer) clearTimeout(run.timer);
44     }
45     runRef.current = null;
46   }, []);
47
48   // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49   useEffect(() => stop, [stop]);
50
51   const submit = useCallback(
52     (rawUri: string) => {
53       const uri = rawUri.trim();
54       if (!uri) {
55         setPhase({ status: "error", error: { code: "invalidUri" } });
56         return;
57       }
58
59       stop(); // supersede any prior run
60       const run: Run = { cancelled: false };
61       runRef.current = run;
62
63       const cid = newCorrelationId();
64       const startedAt = Date.now();
65       setPhase({ status: "submitting" });
66       log("info", "ingest.submit_start", { cid, uri });
67
68       const fail = (error: IngestError, event: string) => {
69         if (run.cancelled) return;
70         setPhase({ status: "error", error });
71         const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72         log(level, event, { cid, uri, code: error.code, detail: error.detail });

```

```

73     };
74
75     const poll = (jobId: string, delay: number) => {
76         run.timer = setTimeout(() => {
77             void (async () => {
78                 if (run.cancelled) return;
79                 let job: JobStatus;
80                 try {
81                     job = await getJob(jobId, cid);
82                 } catch (err) {
83                     fail(classifyError(err), "ingest.poll_error");
84                     return;
85                 }
86                 if (run.cancelled) return;
87
88                 if (job.status === "done") {
89                     const videoId = job.video_id ?? job.result?.video_id ?? null;
90                     // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                     // a success-shaped FAILURE ? never load it, never hang.
92                     if (job.result?.status === "failed" || !videoId) {
93                         fail({ code: "emptyResult" }, "ingest.empty_result");
94                         return;
95                     }
96                     setPhase({ status: "done", videoId });
97                     log("info", "ingest.done", { cid, uri, video_id: videoId });
98                     onLoadedRef.current?.(videoId);
99                     return;
100                 }
101                 if (job.status === "failed") {
102                     fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103                     return;
104                 }
105                 // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106                 setPhase({ status: "polling", progress: job.progress ?? null });
107                 if (Date.now() - startedAt > POLL_CEILING_MS) {
108                     fail({ code: "timeout" }, "ingest.poll_timeout");
109                     return;
110                 }
111                 poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112             })();
113         }, delay);
114     };
115
116     void (async () => {
117         let jobId: string;
118         try {
119             const resp = await processVideo(uri, cid);
120             jobId = resp.job_id;
121         } catch (err) {
122             fail(classifyError(err), "ingest.submit_error");
123             return;
124         }
125         if (run.cancelled) return;
126         log("info", "ingest.queued", { cid, uri, job_id: jobId });
127         setPhase({ status: "polling", progress: null });
128         poll(jobId, POLL_MIN_MS);
129     })();
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {

```

```

135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138   }, [stop]);
139
140   const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143   }, [stop]);
144
145   const busy = phase.status === "submitting" || phase.status === "polling";
146   return { phase, submit, cancel, reset, busy };
147 }
148

```

9. assistant (2026-06-21T16:31:57.884Z)

The handoff (Q4) is confirmed: `useIngestJob` extracts the canonical `video\_id` from the job result (line 89), not the upload's pre-hash id. Now the catalogs and hygiene test.

10. user (2026-06-21T16:31:58.780Z)

```

1    {
2      "app": {
3        "badge": "pick your rallies · alpha",
4        "title": "Pick the rallies for your reel",
5        "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
6        tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
7      },
8      "lang": {
9        "label": "Language",
10       "en": "English",
11       "kn": "?????",
12       "hi": "?????"
13     },
14     "bulk": {
15       "selectAll": "Select all",
16       "clear": "Clear",
17       "invert": "Invert"
18     },
19     "mode": {
20       "demo": "These are sample rallies ? process a video above to work on a real match.",
21       "loading": "Finding the rallies in your video?",
22       "error": "Couldn't load this video's rallies: {error}",
23       "loaded": "Found <b>{count}</b> rallies in your video."
24     },
25     "query": {
26       "placeholder": "e.g. over 10s · top 15 longest · shortest first",
27       "ariaLabel": "Filter rallies by length, count or order",
28       "apply": "Apply",
29       "notAvailable": "not available yet",
30       "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
31       "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
32       rally length instead.",
33       "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
34       "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
35       "try": "Sample queries:",
36       "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
37       <x>experimental</x> (under development)."
38     },
39     "grid": {
40       "empty": "No rallies to show yet.",
41       "rally": "Rally {n}",
42       "inReel": "Rally {n}, in reel",
43       "notInReel": "Rally {n}, not in reel",

```

```

41     "start": "start",
42     "long": "long",
43     "shots": "~{count} shots · experimental",
44     "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
45     "shotsConfidence": " ? confidence: {conf}"
46 },
47     "footer": {
48         "count": "{count, plural, one {# rally} other {# rallies}}",
49         "summaryRest": "selected · {total} total",
50         "reelName": "Reel name",
51         "reelNamePlaceholder": "my-highlights",
52         "build": "Build reel ({count})",
53         "building": "Building?",
54         "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
55         "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
56         "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
57         "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
too short).",
58     },
59     "result": {
60         "ready": "Your reel is ready",
61         "failed": "Build failed",
62         "building": "Stitching your reel?",
63         "buildingReassure": "Joining your selected rallies into one video ? this usually
only takes a moment.",
64         "dismiss": "Dismiss",
65         "download": "Download {filename}"
66     },
67     "build": {
68         "noVideo": "These are sample rallies ? process a real video first to build a reel."
69     },
70     "ingest": {
71         "title": "Process a video",
72         "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
73         "uriPlaceholder": "gs://your-bucket/match.mp4",
74         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
75         "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
76         "submit": "Process video",
77         "submitting": "Submitting?",
78         "cancel": "Cancel",
79         "working": "Processing your video?",
80         "progress": "{progress}",
81         "retry": "Try again",
82         "reassure": "Safe to wait ? KhelSutra keeps working even if your connection is slow;
a long video can take several minutes.",
83         "statusFailed": "Couldn't process",
84         "chooseFile": "Choose a video file",
85         "uploadAria": "Choose a video file to upload",
86         "fileHint": "Pick an .mp4 or .mov from this device ? it uploads, then KhelSutra
finds the rallies.",
87         "orLink": "Or use a cloud link",
88         "uploading": "Uploading? {pct}%"
89     },
90     "errors": {
91         "networkError": "Can't reach KhelSutra. Is it running?",
92         "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
93         "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
94         "timeout": "This is taking longer than expected. The job may still be running ?

```

```

reload with the video link, or try again.",
95     "engineGeneric": "Something went wrong. Please try again."
96 },
97 "selection": {
98     "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
99 }
100 }
101

```

11. user (2026-06-21T16:32:00.044Z)

```

1      // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2      // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3      //     drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4      //     starts enforcing the moment a kn/hi catalog lands (B5/B6).
5      // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      //     aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      //     the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     const LOCALES_DIR = join(import.meta.dirname, "locales");
13     const SRC_DIR = join(import.meta.dirname, "..");
14
15     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16         const keys: string[] = [];
17         for (const [k, v] of Object.entries(obj)) {
18             const key = prefix ? `${prefix}.${k}` : k;
19             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20             else keys.push(key);
21         }
22         return keys.sort();
23     }
24
25     const loadKeys = (locale: string): string[] =>
26         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29         const enKeys = loadKeys("en");
30         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31             .filter((d) => d.isDirectory() && d.name !== "en")
32             .map((d) => d.name);
33
34         it("en is the non-empty source catalog", () => {
35             expect(enKeys.length).toBeGreaterThan(20);
36         });
37
38         for (const loc of others) {
39             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40                 const keys = loadKeys(loc);
41                 const missing = enKeys.filter((k) => !keys.includes(k));
42                 const extra = keys.filter((k) => !enKeys.includes(k));
43                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44             });
45         }
46     });
47

```

```

48     describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49         const FILES = [
50             "App.tsx",
51             "components/QueryBar.tsx",
52             "components/RallyGrid.tsx",
53             "components/SelectionFooter.tsx",
54             "components/CompileResult.tsx",
55             "components/LanguageSwitcher.tsx",
56             "components/IngestPanel.tsx",
57         ];
58         // A hardcoded placeholder / aria-label / title with letters is a bare-string
59         regression ? it
60         // must be an expression ({t(...)}), not a quoted literal.
61         const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"([A-Za-z]{2}[^"]*)"/g;
62         for (const f of FILES) {
63             it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
64                 const src = readFileSync(join(SRC_DIR, f), "utf8");
65                 expect(src.match(BARE_ATTR) ?? []).toEqual([]);
66             });
67         }
68     });
69
70     describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
71         it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google Fonts)", () => {
72             const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73             expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74             expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75         });
76         it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77             const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78             expect(css).toMatch(/Noto Sans Kannada Variable/);
79             expect(css).toMatch(/Noto Sans Devanagari Variable/);
80         });
81     });
82
83     // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
84     // internet) ?
85     // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
86     // CDN/Google Fonts.
87     describe("offline / self-contained ? no external runtime assets", () => {
88         const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
89         const FILES = [
90             "../index.html", // web/index.html
91             "main.tsx",
92             "index.css",
93         ];
94         for (const rel of FILES) {
95             it(`${rel}: no external font/CDN reference`, () => {
96                 expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
97             });
98         }
99         it("web/index.html loads no absolute-URL <link>/<script src> (everything is
100         bundled/local)", () => {
101             const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
102             expect(html).not.toMatch(/(?:href|src)\s*=\s*"(["]https?:\/\//i);
103         });
104     });

```

12. assistant (2026-06-21T16:32:04.253Z)

Now let me verify the engine contract, types, errorText, StatusChip, and the kn/hi catalogs for parity. Let me also check the App.tsx handoff (the `?video=` lift).

13. user (2026-06-21T16:32:05.280Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15     import os
16     import threading
17     import uuid
18     from typing import Any, Dict, Optional
19
20     from fastapi import APIRouter, HTTPException, Request
21     from pydantic import BaseModel
22
23     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24     from backend.utils.paths import safe_output_filename
25
26     router = APIRouter()
27
28     _uploads: Dict[str, Dict[str, Any]] = {}
29     _uploads_lock = threading.Lock()
30
31
32     class UploadInitRequest(BaseModel):
33         filename: str
34         total_size_bytes: Optional[int] = None
35
36
37     class UploadCompleteRequest(BaseModel):
38         total_parts: int
39
40
41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51         None) or ["mp4", "mov"])]
52         return upload_dir, max_bytes, part_bytes, exts
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
59                     os.remove(st["tmp_path"])
60                 except OSError:
61                     pass
62
```

```

63
64 @router.post("/api/upload/init")
65 def upload_init(req: UploadInitRequest):
66     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68     try:
69         name = safe_output_filename(req.filename)
70     except ValueError as e:
71         raise HTTPException(status_code=400, detail=str(e)) from e
72     ext = os.path.splitext(name)[1].lower().lstrip(".")
73     if ext not in exts:
74         raise HTTPException(status_code=400,
75                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77         raise HTTPException(status_code=413,
78                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79     os.makedirs(upload_dir, exist_ok=True)
80     upload_id = uuid.uuid4().hex
81     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82     open(tmp_path, "wb").close()
83     with _uploads_lock:
84         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86           "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89 @router.put("/api/upload/{upload_id}/part/{index}")
90 async def upload_part(upload_id: str, index: int, request: Request):
91     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92     _, max_bytes, part_bytes, _ = _upload_cfg()
93     with _uploads_lock:
94         st = _uploads.get(upload_id)
95         if st is None:
96             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97         if index != st["parts"]:
98             raise HTTPException(status_code=409,
99                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:
110                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                     break
112                 fh.write(chunk)
113         if overflow:
114             _abort_upload(upload_id)
115             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117         with _uploads_lock:
118             st["parts"] += 1
119             st["bytes"] += received
120     return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":

```

```

st["bytes"]}]
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:
140                 _uploads.pop(upload_id, None)
141             video_id = compute_video_id(final_path)
142             return {"path": os.path.abspath(final_path), "video_id": video_id,
143                     "size_bytes": os.path.getsize(final_path),
144                     "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152             if not known:
153                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154             _abort_upload(upload_id)
155             return {"status": "aborted", "upload_id": upload_id}
156

```

14. user (2026-06-21T16:32:06.185Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */

```

```

21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34     status: string;
35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 export interface EngineConfig {
40     stitching?: {
41         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42         */
43         min_shot_count?: number;
44         [key: string]: unknown;
45     };
46     [key: string]: unknown;
47 }
48
49 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
50 /api/jobs/{job_id}. */
51 export interface ProcessResponse {
52     job_id: string;
53     /** Always "queued" at submit. */
54     status: string;
55     /** Relative poll path, e.g. "/api/jobs/<id>". */
56     poll: string;
57 }
58
59 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
60 sequential parts
61 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
62 POST /api/process. */
63 export interface UploadInitResponse {
64     upload_id: string;
65     /** Per-part ceiling in MB ? slice the file into parts this big. */
66     max_part_mb: number;
67     next_part: string;
68 }
69
70 export interface UploadPartResponse {
71     upload_id: string;
72     received_parts: number;
73     received_bytes: number;
74 }
75
76 export interface UploadCompleteResponse {
77     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
78     path: string;
79     video_id: string;
80     size_bytes: number;
81 }
82
83 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
84 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
85 (success|failed).
86 * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
87 shaped

```

```

80      * FAILURE ? the UI must not treat it as a load. */
81      export interface JobStatus {
82          job_id: string;
83          status: "queued" | "running" | "done" | "failed";
84          /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
85          progress?: string | null;
86          /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
87          video_id?: string | null;
88          /** Execution error (worker crash / unexpected) when status === "failed". */
89          error?: string | null;
90          result?: { status?: string; video_id?: string; message?: string } | null;
91      }
92

```

15. user (2026-06-21T16:32:06.914Z)

```

1      // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
surfaces use it:
2      // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
5xx / engine-
3      // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
fetch message.
4      //
5      // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
translated.
6      // Every other code maps to a friendly localized line in the shared `errors.*`
namespace. Kept out of
7      // errors.ts so that module stays i18n-free and unit-testable without a translation
context.
8      import type { TFunction } from "i18next";
9      import type { IngestError } from "../errors";
10
11      export function errorText(t: TFunction, e: IngestError): string {
12          if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13          return t(`errors.${e.code}`);
14      }
15

```

16. user (2026-06-21T16:32:06.960Z)

```

1      // A status indicator that is ALWAYS icon + text ? never icon-only (HCI4D guidance for
non-tech-savvy
2      // users, FRIENDLY_AND_MULTILINGUAL §4: pair every icon with a word, don't rely on color
alone). Used
3      // for the ingest "working / couldn't process" states and the build "stitching / ready /
failed" states.
4      import type { ReactNode } from "react";
5
6      type Tone = "info" | "error";
7
8      const TONE: Record<Tone, string> = {
9          info: "bg-ink/5 text-ink/70",
10         error: "bg-red-100 text-red-800",
11     };
12
13     interface Props {
14         /** A glyph or the <Spinner/> ? decorative; the label carries the meaning for screen
readers. */
15         icon: ReactNode;
16         label: string;
17         tone?: Tone;
18     }
19

```

```

20     export function StatusChip({ icon, label, tone = "info" }: Props) {
21         return (
22             <span className={`inline-flex items-center gap-1.5 rounded-full px-3 py-1 text-sm
font-medium ${TONE[tone]}`}>
23                 <span aria-hidden="true" className="inline-flex">
24                     {icon}
25                 </span>
26                 <span>{label}</span>
27             </span>
28         );
29     }
30
31     /** Indeterminate spinner ? the engine reports coarse phase WORDS, not a percentage, so
progress is a
32     * spinner + the phase label rather than a misleading progress bar. */
33     export function Spinner() {
34         return (
35             <span
36                 className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
37                 aria-hidden="true"
38             />
39         );
40     }
41

```

17. assistant (2026-06-21T16:32:12.360Z)

All key contracts confirmed. The handoff (Q4) is solid: `uploadComplete` returns `{path, video\_id}`, but the client feeds only `path` into `submit`, and `useIngestJob` re-derives the canonical `video\_id` from the job result. The upload's pre-hash `video\_id` is correctly discarded.

Now let me check the kn/hi catalogs for key parity, the App.tsx handoff, and run the actual test suite + tsc/build to verify the "150 green / clean" claim.

18. user (2026-06-21T16:32:13.805Z)

```

web\src\i18n\locales\kn\common.json:84:     "chooseFile": "?????? ???? ?????",
web\src\i18n\locales\kn\common.json:85:     "uploadAria": "????????? ????? ?????? ????? ??????",
web\src\i18n\locales\kn\common.json:86:     "fileHint": "? ? ????????? .mp4 ????? .mov ????? ? ???
????????? ?????????, ???? ?????????? ?????????????? ?????????????.",
web\src\i18n\locales\kn\common.json:87:     "orLink": "???? ?????? ?????? ?????",
web\src\i18n\locales\kn\common.json:88:     "uploading": "????????? ?????????? {pct}%"
web\src\i18n\locales\hi\common.json:84:     "chooseFile": "?????? ???? ??????",
web\src\i18n\locales\hi\common.json:85:     "uploadAria": "?????? ???? ?? ??? ?????? ?????? ??????",
web\src\i18n\locales\hi\common.json:86:     "fileHint": "?? ?????? ?? ?? .mp4 ?? .mov ????? ? ??
????? ?????, ?? ?????????? ??????? ?????? ??????",
web\src\i18n\locales\hi\common.json:87:     "orLink": "?? ?????? ???? ?? ?????? ?????",
web\src\i18n\locales\hi\common.json:88:     "uploading": "????? ?? ??? ??? {pct}%"
web\src\i18n\locales\en\common.json:84:     "chooseFile": "Choose a video file",
web\src\i18n\locales\en\common.json:85:     "uploadAria": "Choose a video file to upload",
web\src\i18n\locales\en\common.json:86:     "fileHint": "Pick an .mp4 or .mov from this device ?
it uploads, then KhelSutra finds the rallies.",
web\src\i18n\locales\en\common.json:87:     "orLink": "Or use a cloud link",
web\src\i18n\locales\en\common.json:88:     "uploading": "Uploading? {pct}%"

```

19. user (2026-06-21T16:32:14.114Z)

```

1     import { useEffect, useMemo, useState } from "react";
2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";

```

```

6     import { SelectionFooter } from "../components/SelectionFooter";
7     import { LanguageSwitcher } from "../components/LanguageSwitcher";
8     import { CompileResult, type BuildState } from "../components/CompileResult";
9     import { IngestPanel } from "../components/IngestPanel";
10    import { runQuery, type ParsedQuery } from "../lib/query";
11    import { floorRisk } from "../lib/floor";
12    import { reelFilename } from "../lib/format";
13    import { classifyError } from "../lib/errors";
14    import { errorText } from "../lib/errorText";
15    import { log, newCorrelationId } from "../lib/log";
16    import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
17    import type { EngineConfig, RallySegment } from "../api/types";
18
19    /** One compact, structured description of how a query was understood (for the trace
log). */
20    function parseTrace(q: ParsedQuery) {
21        return {
22            raw: q.raw,
23            recognized: q.recognized,
24            filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
25            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
26            limit: q.limit ?? null,
27            unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
28        };
29    }
30
31    // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
32    // contract; only honest fields are surfaced. A real session loads from POST /api/query.
33    const DEMO_RALLIES: RallySegment[] = [
34        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
35        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
36        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
37        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
38        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
39        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
40    ];
41
42    type LoadState =
43        | { status: "demo" }
44        | { status: "loading" }
45        | { status: "loaded"; count: number }
46        | { status: "error"; error: string };
47
48    export default function App() {
49        const { t } = useTranslation();
50        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
51        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
52        // without a reload (it also rewrites ?video= so a refresh is stable).
53        const [videoId, setVideoId] = useState<string | null>(
54            () => new URLSearchParams(window.location.search).get("video"),
55        );
56
57        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }

```

```

: { status: "demo" });
60     const [reelName, setReelName] = useState("highlights");
61     const [build, setBuild] = useState<BuildState>({ status: "idle" });
62
63     // Display order is separate from selection: a query can re-sort the cards while the
shared
64     // selection set (sel) stays the single source of truth for what lands in the reel.
65     const sel = useRallySelection(rallies, "all");
66
67     // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
68     useEffect(() => {
69         if (!videoId) return;
70         const cid = newCorrelationId();
71         log("info", "rallies.load_start", { cid, video_id: videoId });
72         // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
73         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
74         setLoadState({ status: "loading" });
75         let alive = true;
76         listRallies(videoId, cid)
77             .then((rs) => {
78                 if (!alive) return;
79                 setRallies(rs);
80                 setOrder(rs);
81                 sel.apply(rs.map((r) => r.id));
82                 setLoadState({ status: "loaded", count: rs.length });
83                 log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
84             })
85             .catch((err) => {
86                 if (!alive) return;
87                 // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
88                 // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
89                 const e = classifyError(err);
90                 setLoadState({ status: "error", error: errorText(t, e) });
91                 log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
92             });
93         return () => {
94             alive = false;
95         };
96         // videoId is stable for the page; sel.apply is a stable callback.
97         // eslint-disable-next-line react-hooks/exhaustive-deps
98     }, [videoId]);
99
100    // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
101    // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
102    const [cfg, setCfg] = useState<EngineConfig | null>(null);
103    useEffect(() => {
104        let alive = true;
105        getConfig()
106            .then((c) => alive && setCfg(c))
107            .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
108        return () => {
109            alive = false;
110        };
111    }, []);
112    const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
113
114    // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
115    const ordinalOf = useMemo(() => {
116        const m = new Map<number, number>();

```



```

117     [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
118     return (id: number) => m.get(id) ?? 0;
119 }, [rallies]);
120
121 // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
122 const onBuild = async () => {
123     if (build.status === "building") return;
124     const cid = newCorrelationId();
125     const filename = reelFilename(reelName);
126     const ids = [...sel.selected];
127     if (!videoId) {
128         log("info", "build.no_video", { cid, selected: ids.length });
129         setBuild({
130             status: "error",
131             error: t("build.noVideo"),
132         });
133         return;
134     }
135     setBuild({ status: "building" });
136     log("info", "build.start", {
137         cid,
138         video_id: videoId,
139         filename,
140         selected: ids.length,
141         total_sec: Math.round(sel.totalSec),
142         below_floor: warning?.atRisk.length ?? 0,
143     });
144     try {
145         const res = await compileReel(videoId, ids, filename, cid);
146         const url = highlightUrl(videoId, filename);
147         setBuild({ status: "done", url, filename });
148         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
149     } catch (err) {
150         // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
151         // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
152         const status = err instanceof ApiError ? err.status : undefined;
153         const e = classifyError(err);
154         setBuild({ status: "error", error: errorText(t, e) });
155         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
156     }
157 };
158
159 // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
160 // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
161 // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
162 const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
163     const cid = newCorrelationId();
164     log("info", "query.parse", { cid, source, ...parseTrace(q) });
165
166     // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
167     // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
168     if (q.unavailable.length > 0) {
169         log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
170     }

```

```

171
172     if (!q.recognized) {
173         log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
174         return; // empty / unparseable ? leave the user's curation untouched
175     }
176
177     try {
178         const { ordered, selectedIds } = runQuery(rallies, q);
179         setOrder(ordered);
180         sel.apply(selectedIds);
181         log("info", "query.apply", {
182             cid,
183             source,
184             filters: q.filters.length,
185             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
186             limit: q.limit ?? null,
187             total: rallies.length,
188             selected: selectedIds.length,
189             selected_ids: selectedIds,
190             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
191         });
192         if (selectedIds.length === 0) {
193             // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
194             log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
195         }
196     } catch (err) {
197         // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
198         log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
199     }
200 };
201
202 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
203 const onToggle = (id: number) => {
204     log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
205     sel.toggle(id);
206 };
207 const onBulk = (op: "select_all" | "clear" | "invert") => {
208     const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
209     log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
210     if (op === "select_all") sel.selectAll();
211     else if (op === "clear") sel.clearAll();
212     else sel.invert();
213 };
214
215 return (
216     <main className="min-h-screen bg-paper text-ink pb-24">
217         <header className="bg-ink text-white">
218             <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
219                 <span className="text-xl font-extrabold tracking-tight">
220                     Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
221                 </span>
222                 <LanguageSwitcher />
223             </div>
224         </header>
225
226         <section className="mx-auto max-w-5xl px-6 py-10">

```

```

227         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
228             {t("app.badge")}
229         </span>
230         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
231         <p className="mt-2 max-w-2xl text-ink/70">
232             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
233         </p>
234
235         /* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
236             path ? process ? load. Once a video_id exists, the existing back-half takes
over. */
237         {!videoId && (
238             <IngestPanel
239                 onLoad={id => {
240                     setVideoId(id);
241                     window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
242                 }}
243             />
244         )}
245
246         <ModeBanner load={loadState} videoId={videoId} />
247
248         <div className="mt-6">
249             <QueryBar onApply={onApplyQuery} />
250         </div>
251
252         <div className="mt-6 flex flex-wrap gap-2 text-sm">
253             <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
254             <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
255             <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
256         </div>
257
258         <div className="mt-5">
259             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
260         </div>
261
262         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
263     </section>
264
265     <SelectionFooter
266         count={sel.count}
267         totalSec={sel.totalSec}
268         reelName={reelName}
269         onReelNameChange={setReelName}
270         onBuild={onBuild}
271         building={build.status === "building"}
272         warning={warning}
273     />
274 </main>
275 );
276 }
277
278 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
279     if (load.status === "demo") {
280         return (

```

```

281         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
282             <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
283         </p>
284     );
285 }
286 if (load.status === "loading") {
287     return (
288         <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
289             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
290         </p>
291     );
292 }
293 if (load.status === "error") {
294     return (
295         <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
296         text-red-800" data-testid="mode-banner">
297             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
298         components={{ code: <code /> }} />
299         </p>
300     );
301 }
302 return (
303     <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
304         <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
305         <strong className="text-ink/70" /> }} />
306     </p>
307 );
308 }
309 }
310 }

```

20. assistant (2026-06-21T16:32:21.352Z)

The handoff chain (Q4) is now fully confirmed end to end: upload `complete` ? `path` only ? `submit(path)` ? `/api/process` ? poll ? `onLoadedRef.current(videoId)` where `videoId` = `job.video\_id` ?? `job.result?.video\_id` (the canonical job id) ? `IngestPanel.onLoaded` ? `App.setVideoId(id)` + `replaceState(?video=<id>)`. The upload's pre-hash `video\_id` is discarded. Correct.

Now let me verify the test/build claims and check the existing IngestPanel test for any regressions in coverage.

21. user (2026-06-21T16:32:22.049Z)

```

web\src\api\client.test.ts
web\src\lib\selection.test.ts
web\src\lib\query.test.ts
web\src\lib\floor.test.ts
web\src\lib\format.test.ts
web\src\components\QueryBar.examples.test.ts
web\src\i18n\i18n.test.tsx
web\src\hooks\useIngestJob.test.ts
web\src\i18n\hygiene.test.ts
web\src\lib\errors.test.ts
web\src\components\IngestPanel.test.tsx
web\src\components\A4.touch.test.tsx
web\src\components\build-error.test.tsx
web\src\lib\errorText.test.ts
web\src\components\A8.status.test.tsx
web\src\api\upload.test.ts
web\src\hooks\useUpload.test.ts
web\src\components\upload.test.tsx

```

22. user (2026-06-21T16:32:22.817Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }
37             // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38             // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39             // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40             if (err.status >= 500) return { code: "networkError" };
41             return { code: "engine", detail: `HTTP ${err.status}` };
42         }
43         return { code: "engine", detail: String(err) };
44     }
45

```

23. assistant (2026-06-21T16:32:26.217Z)

Let me read the upload-related tests to assess regression coverage and the a4/a8 tests to confirm they cover the new file picker.

24. user (2026-06-21T16:32:27.579Z)

```
1      // @vitest-environment jsdom
2      //
3      // T4 end to end through IngestPanel: pick a local file ? it chunks up to /api/upload/*
? the returned
4      // server PATH is fed to POST /api/process ? poll ? onLoaded(video_id). The keystone for
alpha users
5      // who have a FILE, not a gs:// URI. Also pins that the file picker is present +
labelled (a11y/A4).
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, cleanup, fireEvent, act } from "@testing-library/react";
8      import i18n from "../i18n";
9      import { IngestPanel } from "../IngestPanel";
10
11      // Flush promise microtasks (the upload/process awaits) under fake timers ? waitFor
can't advance them.
12      async function flush(times = 15) {
13          for (let i = 0; i < times; i += 1) {
14              await act(async () => {
15                  await vi.advanceTimersByTimeAsync(0);
16              });
17          }
18      }
19
20      const MB = 1024 * 1024;
21      const ok = (j: unknown) => ({ ok: true, status: 200, json: async () => j, text: async ()
=> JSON.stringify(j) });
22
23      let processBody: { video_path?: string } | null = null;
24
25      function uploadThenProcessFetch() {
26          processBody = null;
27          return vi.fn(async (url: string | URL, init?: RequestInit) => {
28              const u = String(url);
29              const m = init?.method ?? "GET";
30              if (u === "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1,
next_part: "/api/upload/up1/part/0" });
31              if (u.includes("/api/upload/up1/part/")) return ok({ upload_id: "up1",
received_parts: 1, received_bytes: (init?.body as Blob).size });
32              if (u.endsWith("/complete")) return ok({ path: "/srv/uploads/match.mp4", video_id:
"PRE", size_bytes: 1 });
33              if (u === "/api/process" && m === "POST") {
34                  processBody = JSON.parse(init!.body as string);
35                  return ok({ job_id: "j", status: "queued", poll: "/api/jobs/j" });
36              }
37              if (u.startsWith("/api/jobs/")) return ok({ job_id: "j", status: "done", video_id:
"VIDFINAL", result: { status: "success", video_id: "VIDFINAL" } });
38              throw new Error(`unexpected fetch ${m} ${u}`);
39          }) as unknown as typeof fetch;
40      }
41
42      const videoFile = (bytes: number) => new File([new Uint8Array(bytes)], "match.mp4", {
type: "video/mp4" });
43
44      const pickFile = (file: File) => {
45          const input = screen.getByLabelText("Choose a video file to upload");
46          Object.defineProperty(input, "files", { value: [file], configurable: true });
47          fireEvent.change(input);
48      };
49
50      beforeEach(async () => {
51          window.localStorage.clear();
52          await i18n.changeLanguage("en");
53      });
```

```

54
55   afterEach(() => {
56     cleanup();
57     vi.restoreAllMocks();
58     vi.unstubAllGlobals();
59     vi.useRealTimers();
60   });
61
62   describe("T4: in-UI file upload", () => {
63     it("offers a labelled file picker (present for a keyboard/SR user)", () => {
64       vi.stubGlobal("fetch", vi.fn());
65       render(<IngestPanel onLoaded={vi.fn()} />);
66       expect(screen.getByLabelText("Choose a video file to upload")).toBeTruthy();
67       expect(screen.getByText("Choose a video file")).toBeTruthy();
68     });
69
70     it("uploads the picked file, feeds the server path to /api/process, and loads the
resulting video_id", async () => {
71       vi.useFakeTimers();
72       vi.stubGlobal("fetch", uploadThenProcessFetch());
73       const onLoaded = vi.fn();
74       render(<IngestPanel onLoaded={onLoaded} />);
75
76       pickFile(videoFile(0.3 * MB));
77
78       // Flush the upload chain (init ? part ? complete) + the process submit ? schedules
the poll.
79       await flush();
80       // The uploaded server PATH (not the URI) is what gets processed.
81       expect(processBody?.video_path).toBe("/srv/uploads/match.mp4");
82
83       // Poll tick ? done ? the job's canonical video_id is handed up.
84       await act(async () => {
85         await vi.advanceTimersByTimeAsync(1000);
86       });
87       await flush();
88       expect(onLoaded).toHaveBeenCalledWith("VIDFINAL");
89     });
90   });
91

```

25. user (2026-06-21T16:32:27.674Z)

```

1    // @vitest-environment jsdom
2    //
3    // A4 ? non-tech-savvy mobile UX. Two guarantees a phone user depends on:
4    //   1. TOUCH TARGETS ? 44px (WCAG 2.5.5 / platform HIG) on every primary control ?
pinned via the
5    //     Tailwind min-h-11 (=2.75rem=44px) / h-11 class so a future "tighten the padding"
can't
6    //     silently shrink a tap target below the floor.
7    //   2. The "not available yet" REASON is VISIBLE text, not a hover-only title ? a touch
device never
8    //     surfaces `title=`, so the explanation must render in the DOM for a phone user to
read it.
9    import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
10   import { render, screen, cleanup, fireEvent } from "@testing-library/react";
11   import i18n from "../i18n/index";
12   import App from "../App";
13   import { QueryBar } from "../QueryBar";
14
15   // 44px floor: min-h-11 (buttons/inputs) or an explicit h-11 square (the ? dismiss).
16   const TAP_44 = /\b(?:min-h-11|h-11)\b/;
17
18   beforeEach(async () => {

```

```

19     vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
20     window.localStorage.clear();
21     await i18n.changeLanguage("en");
22   });
23
24   afterEach(() => {
25     cleanup();
26     vi.unstubAllGlobals();
27   });
28
29   describe("A4: touch targets ? 44px on primary controls", () => {
30     it("the always-visible buttons carry a ?44px tap class", () => {
31       render(<App />);
32       for (const name of [
33         "Select all",
34         "Clear",
35         "Invert",
36         "Apply", // QueryBar submit
37         "Process video", // IngestPanel submit
38         "English", // LanguageSwitcher
39         "?????",
40         "??????",
41       ]) {
42         const btn = screen.getByRole("button", { name });
43         expect(btn.className, `${name} must be ?44px`).toMatch(TAP_44);
44       }
45       // The Build button label includes the live selection count ("Build reel (6)").
46       expect(screen.getByRole("button", { name: /Build reel/
47     }).className).toMatch(TAP_44);
48   });
49
50   it("the text inputs carry a ?44px tap class", () => {
51     render(<App />);
52     expect(screen.getByRole("textinput", { name: "Filter rallies by length, count or
53     order" }).className).toMatch(TAP_44);
54     expect(screen.getByRole("textinput", { name: "Reel name"
55     }).className).toMatch(TAP_44);
56     expect(
57       screen.getByRole("textinput", { name: "Video location ? a gs:// bucket URL or a path
58       on the engine host" }).className,
59     ).toMatch(TAP_44);
60   });
61
62   it("the sample-query chips are tappable (?44px)", () => {
63     render(<App />);
64     expect(screen.getByRole("button", { name: "over 10s" }).className).toMatch(TAP_44);
65     expect(screen.getByRole("button", { name: "top 15 longest"
66     }).className).toMatch(TAP_44);
67   });
68
69   describe("A4: per-card shot-count confidence is visible, not hover-only", () => {
70     it("renders the confidence inline on the chip (phones never see a title=)", () => {
71       const { container } = render(<App />); // demo rallies include a High-confidence
72       card
73       expect(container.textContent).toContain("confidence: High");
74     });
75   });
76
77   describe("A4: the 'not available yet' reason is visible, not hover-only", () => {
78     const HELP = "Shot type, players and score aren't available yet ? filter by rally
79     length instead.";
80
81     it("hidden until an unsupported concept is typed, then shown as plain text", () => {
82       render(<QueryBar onApply={() => {}} />);

```



```

77         expect(screen.queryByText(HELP)).toBeNull(); // nothing typed ? no help line
78         fireEvent.change(screen.getByRole("textbox", { name: "Filter rallies by length,
count or order" })), {
79             target: { value: "smashes" }, // a shot-type token ? flagged "not available yet"
80         });
81         expect(screen.getByText(HELP)).toBeTruthy(); // the reason is now a visible DOM node
82     });
83 });
84

```

26. assistant (2026-06-21T16:32:31.771Z)

Now let me run the actual test suite + tsc + build to verify the claims, and confirm there are no regressions.

27. user (2026-06-21T16:32:38.944Z)

```

{"ts":"2026-06-21T16:32:38.578Z","level":"error","event":"api.network_error","cid":"c693e1c2-2064-414a-b041-13f963be77b6","method":"GET","url":"/api/config","error":"Error: no engine in test"}

```

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B5:
switching to Kannada renders the kn catalog (Kannada copy, not English)
{"ts":"2026-06-21T16:32:38.578Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

```

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:
switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T16:32:38.599Z","level":"info","event":"api.request","cid":"0ea34d3c-3469-46cb-a49a-e24047923e8b","method":"GET","url":"/api/config"}

```

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:
switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T16:32:38.625Z","level":"error","event":"api.network_error","cid":"0ea34d3c-3469-46cb-a49a-e24047923e8b","method":"GET","url":"/api/config","error":"Error: no engine in test"}

```

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:
switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T16:32:38.625Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

```

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a
locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T16:32:38.641Z","level":"info","event":"api.request","cid":"8dbd96a2-138b-4f1f-be06-fe960b4102d7","method":"GET","url":"/api/config"}

```

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a
locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T16:32:38.644Z","level":"error","event":"api.network_error","cid":"8dbd96a2-138b-4f1f-be06-fe960b4102d7","method":"GET","url":"/api/config","error":"Error: no engine in test"}

```

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a
locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T16:32:38.644Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

```

stdout | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T16:32:38.662Z","level":"info","event":"api.request","cid":"471e4f8a-

```

```
8f88-4b49-9916-9b14631dc55a","method":"GET","url":"/api/config"}
```

```
stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-
21T16:32:38.665Z","level":"error","event":"api.network_error","cid":"471e4f8a-8f88-4b49-9916-
9b14631dc55a","method":"GET","url":"/api/config","error":"Error: no engine in test"}
```

```
stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-
21T16:32:38.665Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}
```

```
? src/i18n/i18n.test.tsx (13 tests)
692ms
```

```
? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 364ms
```

```
Test Files 18 passed (18)
Tests 150 passed (150)
Start at 22:02:34
Duration 4.27s (transform 842ms, setup 238ms, collect 5.47s, tests 3.07s,
environment 20.38s, prepare 4.42s)
```

28. assistant (2026-06-21T16:32:41.996Z)

150 tests pass. Now tsc + build, and lint.

29. user (2026-06-21T16:32:49.478Z)

```
=== TSC DONE ===
vite v6.4.3 building for production...
transforming...
? 91 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html
  0.40 kB ? gzip: 0.27 kB
dist/assets/noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
 13.85 kB
dist/assets/noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
 13.99 kB
dist/assets/noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
 25.38 kB
dist/assets/noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
 27.58 kB
dist/assets/noto-sans-kannada-kannada-wght-normal-DuwB-P8d.woff2
 90.80 kB
dist/assets/noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
121.19 kB
dist/assets/index-CigX8twa.css
 22.32 kB ? gzip: 5.11 kB
dist/assets/index-BLf7UsQJ.js
356.73 kB ? gzip: 111.88 kB
? built in 1.18s
```

30. assistant (2026-06-21T16:32:52.749Z)

tsc clean, build clean. Now let me run lint to match the repo's CI.

31. user (2026-06-21T16:32:53.296Z)

```

"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "test": "vitest run"
},
"dependencies": {
  "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
  "@fontsource-variable/noto-sans-kannada": "^5.2.9",
  "i18next": "^26.3.1",
  "i18next-browser-languagedetector": "^8.2.1",
  "i18next-icu": "^2.4.3",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-i18next": "^17.0.8"
},
"devDependencies": {
  "@tailwindcss/vite": "^4.0.0",
  "@testing-library/dom": "^10.4.1",
  "@testing-library/react": "^16.3.2",

```

32. assistant (2026-06-21T16:32:58.067Z)

No lint script in this project (no eslint configured). tsc + tests + build are the gates, all green. Let me do a focused review of the remaining a11y/UX concerns. Let me check the a8 status test and the useUpload test to confirm progressbar + handoff coverage, and verify the disabled-input behavior precisely.

33. user (2026-06-21T16:32:58.563Z)

```

1      // @vitest-environment jsdom
2      //
3      // A8 ? honest expectations for non-tech users on slow uplinks: background-processing
reassurance and
4      // status shown as icon+TEXT chips (never icon-only), on BOTH the ingest progress and
the build state.
5      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
6      import { render, screen, cleanup, fireEvent, act } from "@testing-library/react";
7      import i18n from "../i18n";
8      import { CompileResult } from "../CompileResult";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     beforeEach(async () => {
13       window.localStorage.clear();
14       await i18n.changeLanguage("en");
15     });
16
17     afterEach(() => {
18       cleanup();
19       vi.restoreAllMocks();
20       vi.unstubAllGlobals();
21       vi.useRealTimers();
22     });
23
24     describe("A8: build state shows status + reassurance (CompileResult)", () => {
25       it("building ? a 'stitching?' chip + a reassurance line (and nothing to dismiss yet)",
() => {
26         render(<CompileResult build={{ status: "building" }} onDismiss={() => {}} />);
27         expect(screen.getByText("Stitching your reel?")).toBeTruthy();
28         expect(screen.getByText(/only takes a moment/i)).toBeTruthy();
29         expect(screen.queryByRole("button", { name: "Dismiss" })).toBeNull();
30       });
31

```

```

32     it("done ? a ? icon paired with the 'ready' text (icon+text, not icon-only)", () => {
33         render(<CompileResult build={{ status: "done", url: "/x", filename: "r.mp4" }}
onDismiss={() => {}} />);
34         const h = screen.getByRole("heading", { level: 2 });
35         expect(h.textContent).toContain("?");
36         expect(h.textContent).toContain("Your reel is ready");
37     });
38
39     it("error ? a ? icon paired with 'failed' + the message", () => {
40         render(<CompileResult build={{ status: "error", error: "boom" }} onDismiss={() =>
41         {}} />);
42         const h = screen.getByRole("heading", { level: 2 });
43         expect(h.textContent).toContain("?");
44         expect(screen.getByText("boom")).toBeTruthy();
45     });
46
47     describe("A8: ingest progress reassurance + status chips (IngestPanel)", () => {
48         const httpOk = (s: number, j: unknown) => ({ ok: s >= 200 && s < 300, status: s, json:
49         async () => j, text: async () => JSON.stringify(j) });
50         function fetchSeq(process: unknown, jobs: JobStatus[]) {
51             let i = 0;
52             return vi.fn(async (url: string | URL, init?: RequestInit) => {
53                 const u = String(url);
54                 if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
55                 if (u.startsWith("/api/jobs/")) {
56                     const b = jobs[Math.min(i, jobs.length - 1)];
57                     i += 1;
58                     return httpOk(200, b);
59                 }
60                 throw new Error(`unexpected fetch ${u}`);
61             }) as unknown as typeof fetch;
62
63             it("while processing ? the working chip + 'safe to wait' reassurance", async () => {
64                 vi.useFakeTimers();
65                 vi.stubGlobal("fetch", fetchSeq(httpOk(202, { job_id: "j", status: "queued" }), [
66                     { job_id: "j", status: "running", progress: "segmenting" },
67                 ]));
68                 render(<IngestPanel onLoaded={vi.fn()} />);
69                 fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
70                 "gs://b/x.mp4" } });
71                 fireEvent.click(screen.getByRole("button", { name: /process video/i }));
72                 await act(async () => {
73                     await vi.advanceTimersByTimeAsync(0); // submit ? polling
74                 });
75                 await act(async () => {
76                     await vi.advanceTimersByTimeAsync(1000); // running tick ? progress word
77                 });
78                 expect(screen.getByText(/processing your video/i)).toBeTruthy();
79                 expect(screen.getByText(/safe to wait/i)).toBeTruthy();
80             });
81
82             it("on failure ? a ? 'Couldn't process' status chip alongside the friendly message",
83             async () => {
84                 vi.stubGlobal("fetch", vi.fn(async () => {
85                     throw new Error("offline");
86                 }) as unknown as typeof fetch);
87                 render(<IngestPanel onLoaded={vi.fn()} />);
88                 fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
89                 "gs://b/x.mp4" } });
90                 fireEvent.click(screen.getByRole("button", { name: /process video/i }));
91                 expect(await screen.findByText("Couldn't process")).toBeTruthy();
92                 expect(await screen.findByText(/is it running/i)).toBeTruthy();
93             });

```

```
91     });
92
```

34. user (2026-06-21T16:32:59.056Z)

```
1      // @vitest-environment jsdom
2      //
3      // The upload state machine (T4): start(file) ? uploading(%) ? hand the server PATH to
onComplete |
4      // error. Pins: an empty file is rejected before any network call, a happy upload calls
onComplete
5      // with the returned path and returns to idle, and an engine rejection surfaces a
classified error.
6      import { describe, it, expect, vi, afterEach } from "vitest";
7      import { renderHook, act, waitFor } from "@testing-library/react";
8      import { useUpload } from "../useUpload";
9
10     type FetchFn = typeof fetch;
11     const MB = 1024 * 1024;
12     const ok = (j: unknown) => ({ ok: true, status: 200, json: async () => j, text: async ()
=> JSON.stringify(j) });
13     const httpErr = (status: number, detail: string) => ({
14         ok: false, status, json: async () => ({ detail }), text: async () => JSON.stringify({
detail }),
15     });
16
17     function uploadFetch(opts: { partFailsAt?: number } = {}): FetchFn {
18         return vi.fn(async (url: string | URL, init?: RequestInit) => {
19             const u = String(url);
20             const m = init?.method ?? "GET";
21             if (u === "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1,
next_part: "/api/upload/up1/part/0" });
22             const part = u.match(/\/part\/(\d+)\$/);
23             if (part && m === "PUT") {
24                 if (opts.partFailsAt === Number(part[1])) return httpErr(413, "Part exceeds the
per-part limit");
25                 return ok({ upload_id: "up1", received_parts: 1, received_bytes: (init?.body as
Blob).size });
26             }
27             if (u.endsWith("/complete")) return ok({ path: "/srv/uploads/match.mp4", video_id:
"VID", size_bytes: 1 });
28             if (m === "DELETE") return ok({ status: "aborted" });
29             throw new Error(`unexpected ${m} ${u}`);
30         }) as unknown as FetchFn;
31     }
32
33     const videoFile = (bytes: number, name = "match.mp4") => new File([new
Uint8Array(bytes)], name, { type: "video/mp4" });
34
35     afterEach(() => {
36         vi.restoreAllMocks();
37         vi.unstubAllGlobals();
38     });
39
40     describe("useUpload", () => {
41         it("rejects an empty file before any network call", () => {
42             const f = vi.fn();
43             vi.stubGlobal("fetch", f as unknown as FetchFn);
44             const onComplete = vi.fn();
45             const { result } = renderHook(() => useUpload(onComplete));
46
47             act(() => result.current.start(videoFile(0)));
48
49             expect(result.current.phase).toEqual({ status: "error", error: { code: "invalidUri"
} });
50         });
51     });
```

```

50     expect(f).not.toHaveBeenCalled();
51     expect(onComplete).not.toHaveBeenCalled();
52   });
53
54   it("uploads a file and hands the returned server path to onComplete", async () => {
55     vi.stubGlobal("fetch", uploadFetch());
56     const onComplete = vi.fn();
57     const { result } = renderHook(() => useUpload(onComplete));
58
59     act(() => result.current.start(videoFile(0.3 * MB)));
60     await waitFor(() =>
expect(onComplete).toHaveBeenCalled("srv/uploads/match.mp4"));
61     expect(result.current.phase.status).toBe("idle"); // handed off ? the engine phase
takes over
62   });
63
64   it("surfaces a classified error when a part is rejected", async () => {
65     vi.stubGlobal("fetch", uploadFetch({ partFailsAt: 0 }));
66     const onComplete = vi.fn();
67     const { result } = renderHook(() => useUpload(onComplete));
68
69     act(() => result.current.start(videoFile(0.3 * MB)));
70     await waitFor(() => expect(result.current.phase.status).toBe("error"));
71     expect(onComplete).not.toHaveBeenCalled();
72     // a 413 with a {detail} body classifies as an engine error (its message survives)
73     expect(result.current.phase).toMatchObject({ status: "error", error: { code:
"engine" } });
74   });
75 });
76

```

35. assistant (2026-06-21T16:33:12.840Z)

I now have a complete picture. Let me work through each review dimension precisely.

****Q1 ? File picker a11y:****

- Keyboard reachable: the native `<input type="file">` is `sr-only` (not `display:none`/`hidden`), so it remains focusable and in the tab order. It's inside a `<label>`, so the visible styled label text triggers it. Correct. ?
- Screen-reader labelled: `aria-label={t("ingest.uploadAria")}` ("Choose a video file to upload"). ?
- ?44px (A4): the `<label>` carries `min-h-11` (=44px). But this is on the **label wrapper**, and the focusable element is the sr-only input ? focus ring would be on the invisible input, not the visible label. This is a real but minor a11y nuance (see findings). The visible target is ?44px. ? on size.
- No-bare-strings: `aria-label`, `placeholder`, `title` all route through `t()`. The `BARE_ATTR` regex won't match. The `accept=".mp4,.mov,..."` attribute is not in the regex's attr list, so it's fine. ?

****Q2 ? Progress:**** `role=progressbar` + `aria-valuenow/min/max` correct, determinate for upload (real byte %), spinner-only indeterminate for engine phase. Honest. ? One nuance: the progressbar lacks an accessible name (`aria-label`/`aria-labelledby`), but the adjacent StatusChip text "Uploading? N%" carries the meaning via aria-live. Minor.

****Q3 ? Both paths:**** gs:// preserved (a8/a4 tests still drive it). During busy both inputs `disabled={busy}` where `busy = ingestBusy || upload.busy`. Cancel shown. Unified error via `errorState` mutual-exclusive. `resetAll` resets both. ?

****Q4 ? Handoff:**** canonical job video_id, confirmed end to end. The `upload.test.tsx` explicitly pins this: complete returns `video_id:"PRE"`, job returns `VIDFINAL`, and `onLoaded` is called with `VIDFINAL`. Excellent regression guard. ?

****Q5 ? Copy honesty:**** Let me scrutinize. There is one issue worth flagging.

Let me verify one detail about the focus-visibility of the sr-only input and whether there's a

focus style, and check the `accept` attribute robustness.

36. user (2026-06-21T16:33:13.996Z)

```
web\src\components\IngestPanel.tsx:72:          className="sr-only"
```

37. assistant (2026-06-21T16:33:22.413Z)

Confirmed: the file ``<input>`` is `sr-only` and the ``<label>`` has no `focus-within:` style, so when a keyboard user tabs to the file picker, the focus lands on the visually-hidden input with no visible focus indicator on the styled label. The button looks unfocused even though it is focused. That's a WCAG 2.4.7 (Focus Visible) gap ? the only substantive a11y finding.

Let me confirm the `min-h-11` Tailwind mapping (11 = 2.75rem = 44px) is correct and that the picker label is included ? it is (`min-h-11` on line 61). Also note the gs:// `orLink` copy and `fileHint` for honesty.

Everything checks out. Let me also verify there's no regression where the file input remains tabbable while disabled (it has `disabled={busy}` AND `pointer-events-none` on the label during busy ? disabled inputs are removed from the tab order, good).

One more honesty check on copy: `fileHint` says "Pick an .mp4 or .mov from this device" but the `accept` attribute also allows `video/mp4,video/quicktime` MIME types ? consistent with .mp4/.mov, no over-promise. The engine's `allowed\_upload\_extensions` default is `["mp4","mov"]`, matching. Good ? no mismatch.

Let me compose the structured findings.

38. user (2026-06-21T16:33:50.292Z)

Output does not match required schema: root: must have required property 'findings'

39. assistant (2026-06-21T16:33:54.212Z)

I had a malformed call. Let me resubmit cleanly.

40. user (2026-06-21T16:34:15.281Z)

Output does not match required schema: root: must have required property 'findings'

41. assistant (2026-06-21T16:34:19.693Z)

The schema requires `findings` at the root. My calls include it, but the harness is rejecting ? likely the nested XML-style encoding isn't being parsed as JSON. Let me retry with a minimal, clearly-structured payload.

42. user (2026-06-21T16:34:35.661Z)

Output does not match required schema: root: must have required property 'findings'

43. assistant (2026-06-21T16:34:40.012Z)

The harness isn't accepting `findings` as a JSON string. The schema declares `findings` as an array of objects ? I need to pass it as a native array, with each finding as a parameter object. Let me retry using proper nested parameter structure.

44. user (2026-06-21T16:34:52.538Z)

Output does not match required schema: root: must have required property 'findings'

45. assistant (2026-06-21T16:34:56.773Z)

The tool input is being parsed as JSON, and my parameters need to be valid JSON objects/arrays.

The XML-style parameter encoding for the `summary` (string) and `findings` (array) needs the array as a JSON array literal. Let me try a minimal call first with an empty findings array to confirm the encoding, then build up.

46. user (2026-06-21T16:34:57.117Z)

Structured output provided successfully

47. assistant (2026-06-21T16:35:01.086Z)

The empty array worked ? the issue was that prose/YAML inside ``<findings>`` wasn't valid JSON. The findings must be a JSON array literal. Let me submit the real answer with proper JSON.

48. user (2026-06-21T16:35:19.851Z)

Output does not match required schema: root: must have required property 'findings'

49. assistant (2026-06-21T16:35:25.128Z)

Interesting ? the empty `[]` passed but the populated array fails. The difference must be that the long JSON string with special characters (the `?` ellipsis, curly quotes, em-dashes) is breaking the parse, or the embedded `{t('...')}` braces. Let me simplify: avoid any characters that could confuse the parser, and use plain ASCII only.

50. user (2026-06-21T16:35:38.255Z)

Output does not match required schema: root: must have required property 'findings'

51. assistant (2026-06-21T16:35:42.564Z)

Still failing with ASCII-only. The empty array `[]` worked but a populated array fails ? this suggests the parser fails when ``<findings>`` contains object content, possibly because of the nested `{}` braces being interpreted. Let me test with a single minimal finding to isolate.

52. user (2026-06-21T16:35:43.270Z)

Structured output provided successfully

53. assistant (2026-06-21T16:35:48.548Z)

That worked. The single minimal finding with spaces after colons passed. The earlier failures were likely due to a parse issue with my larger payload ? possibly the `OR` text, parentheses, or length. Let me build the full payload incrementally, keeping the exact JSON formatting style that just worked (spaces after colons), and avoid parentheses-heavy detail. Let me submit all three findings.

54. user (2026-06-21T16:36:06.969Z)

Output does not match required schema: root: must have required property 'findings'

55. assistant (2026-06-21T16:36:11.584Z)

The single small finding worked, but the large payload fails. The likely culprit is the very long `summary` combined with the findings, or a specific character. The "test2" + 1-finding call succeeded. Let me keep the findings identical to the working format but shorten the summary drastically and shorten each detail to avoid whatever length/char threshold is tripping the parser.

56. user (2026-06-21T16:36:23.571Z)

Output does not match required schema: root: must have required property 'findings'

57. assistant (2026-06-21T16:36:27.907Z)

Three findings fail but one finding works. The breaking factor is the number of findings (or total payload size). Let me test with two findings to narrow it down.

58. user (2026-06-21T16:36:29.012Z)

Structured output provided successfully

59. assistant (2026-06-21T16:36:35.307Z)

Two findings work with short content. So the failure is content-length related, not count. The `location` field or long `detail` strings are pushing total size past a limit, OR a specific character. Let me add the third short finding and the real (longer) content one field at a time. Let me try all three with short details first.

60. user (2026-06-21T16:36:38.650Z)

Structured output provided successfully

61. assistant (2026-06-21T16:36:43.152Z)

Three findings with full metadata work fine when details are short. So the problem is the ****total payload length**** (long `summary` + long `detail`/`suggestion` strings). The schema validation error message is misleading ? it's actually a size/truncation issue. Let me keep the summary concise and trim each detail to its essential content while preserving the load-bearing facts.

62. user (2026-06-21T16:36:58.398Z)

Output does not match required schema: root: must have required property 'findings'

63. assistant (2026-06-21T16:37:03.015Z)

Still failing. The three-short-findings call passed. The difference now is the long `summary` AND long details together. Let me isolate: keep the proven-short summary but use the full-length findings.

64. user (2026-06-21T16:37:10.636Z)

Structured output provided successfully

65. assistant (2026-06-21T16:37:18.630Z)

The structured output was accepted. The earlier failures were a total-payload-size limit (the long summary combined with the long findings exceeded it).

Review complete. T4 is solid and ships the North Star keystone correctly ? verified locally (150 web tests pass, tsc clean, vite build clean; this project has no lint script). All five review dimensions hold up, and the handoff (Q4) is the standout: the upload's pre-hash `video\_id` is correctly discarded and the canonical job `video\_id` flows through, pinned by `upload.test.tsx`.

The only substantive item is one ****minor**** a11y gap: the file picker has no visible keyboard-focus indicator (WCAG 2.4.7) because the focusable element is the `sr-only` input and the styled label lacks a `focus-within:` style. Plus two nits (unnamed progressbar; Cancel/percent-chip inside the aria-live region). Nothing blocking.